

Keeping a Local Claude Code Session Always Live

Four options, ranked by effort — from a five-minute tmux fix to a dedicated headless box — plus a priced Windows build, the prerequisites, the failure modes, and copy-paste service files that make it survive a reboot.

THE SHORT VERSION

You almost certainly do not need to buy hardware. The thing that ends a local session is the **claude** process exiting — not the absence of a dedicated machine. Sleep and network drops are already handled: Claude Code queues your messages and reconnects on its own when the machine comes back. Run the session inside **tmux**, stop the machine sleeping, and you have solved most of the problem for free. Buy a mini PC only once you have proved the real constraint is that your only computer leaves the house.

Which option is yours?

If this is you	Go to	Effort
I have a desktop or a laptop that mostly stays home, and I want to check in from my phone	Option 1 — tmux and no-sleep (p. 2)	5 min
I want several sessions at once, started from my phone, without touching the terminal first	Option 2 — server mode (p. 3)	15 min
My only machine is a laptop that travels with me, and I want true 24/7	Option 3 — a headless box (p. 3)	1–2 hrs plus hardware
I want none of this, and the work does not need my local files or LAN	Option 4 — cloud sessions (p. 6)	0 min
I am on Windows, and I want the hardware priced	Windows: the headless build (p. 4)	Reference

Before anything: the prerequisites

Remote Control is the feature all of this rests on. It connects `claude.ai/code` or the Claude mobile app to a session running on *your* machine — execution and filesystem access stay local, and the web and phone are a window onto that. Check these first; several of them fail quietly on a freshly provisioned box.

Requirement	Detail
Plan	Pro, Max, Team, or Enterprise. API keys are not supported. On Team and Enterprise an Owner must first enable the Remote Control toggle in Claude Code admin settings.
Sign-in	Run <code>cclaude</code> and use <code>/login</code> to sign in through <code>claude.ai</code> .

Requirement	Detail
API endpoint	Must be <code>api.anthropic.com</code> . Not available on Amazon Bedrock, Google Cloud's Agent Platform, or Microsoft Foundry, nor when <code>ANTHROPIC_BASE_URL</code> points at an LLM gateway or proxy, nor behind an enterprise Claude apps gateway.
Env vars to unset	<code>DISABLE_TELEMETRY</code> , <code>DO_NOT_TRACK</code> , <code>CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC</code> , <code>DISABLE_GROWTHBOOK</code> . Each one disables the feature-flag evaluation Remote Control depends on. Check your shell profile <i>and</i> the env block of <code>settings.json</code> .
Workspace trust	Run <code>claude</code> in the project directory once, interactively, and accept the trust dialog. The dialog never saves trust for your home directory, so always start from a project directory — not from <code>\$HOME</code> .

Option 1 — tmux, and stop the machine sleeping

Five minutes, no new hardware, and it solves the common case. This is where everyone should start. The documentation is explicit about the mechanism: to keep a session running on a remote machine after you disconnect from SSH, start it inside `tmux` or `screen`.

```
tmux new -s claude
claude --remote-control "my-project"      # or --rc
# Ctrl-b then d to detach; the session keeps running
# tmux attach -t claude to come back
```

Already mid-session in a terminal? Just type `/remote-control` (or `/rc`) and that conversation becomes reachable from your other devices. Press the spacebar in the terminal for a QR code, or open the Claude app, tap Code, and pick the session from the list.

Stop the machine sleeping

Sleep is the one thing that genuinely interrupts you. Pick your platform:

```
# macOS – for the length of one session
caffeinate -s -i tmux new -s claude

# macOS – permanently, for a machine that should never sleep
sudo pmset -a sleep 0 disablesleep 1

# Linux – mask the sleep targets outright
sudo systemctl mask sleep.target suspend.target hibernate.target

# Linux laptop, lid closed: in /etc/systemd/logind.conf
#   HandleLidSwitch=ignore
#   HandleLidSwitchExternalPower=ignore
# then: sudo systemctl restart systemd-logind
```

WHAT YOU DO NOT NEED TO WORRY ABOUT

If the laptop sleeps anyway, or the Wi-Fi drops, Claude Code reconnects by itself once the machine is back online. While the connection rebuilds it queues your messages, permission prompts, and status updates from subagents and workflows, then delivers them. An interactive session retries for as long as the outage lasts. The failure mode to design against is the process *exiting*, which is exactly what `tmux` prevents.

Option 2 — server mode, the feature built for this

Instead of one interactive conversation, the `claude remote-control` process parks in your terminal and waits for connections, creating sessions on demand from your phone or browser. Any machine running it shows up as a device card at the top of the Code tab in the Claude app. This is the shape you want on an always-on machine.

```
cd ~/code/my-project
claude remote-control --spawn worktree --capacity 8

# First run asks: Enable Remote Control? (y/n) -- one-time confirmation
# Press spacebar for a QR code, 'w' to toggle worktree mode at runtime
```

Flags worth knowing (all of them go *after* `remote-control`):

Flag	What it does
--spawn worktree	Each on-demand session gets its own git worktree, so parallel sessions do not edit the same files. The alternative, <code>same-dir</code> , is the default and shares one directory; session serves exactly one session and rejects the rest.
--capacity N	Maximum concurrent sessions. Default is 32. Cannot be combined with <code>--spawn=session</code> .
-c, --continue	Bring back the session the last server in this directory used, instead of creating a new one. The flag to put in your service file. Requires v2.1.200 or later.
--permission-mode	Starting permission mode for the server's sessions, e.g. <code>acceptEdits</code> . Worth thinking about for a machine you will drive from a phone.
--name	A session title you will recognise in the list at <code>claude.ai/code</code> . Without it, names are generated from the hostname, like <code>myhost-graceful-unicorn</code> .
--sandbox	Filesystem and network isolation. Off by default. Consider it for an unattended box.

TWO SHARP EDGES IN SERVER MODE

1. If the machine is awake but cannot reach the network, server mode gives up after roughly ten minutes and the process exits — unlike an interactive session, which retries indefinitely. That is the single strongest argument for the auto-restart service on page 5. **2.** Do not pass global `claude` flags *before* the `remote-control` subcommand: they are not carried over to the sessions the server creates, and Claude Code refuses to start rather than silently dropping them.

A session that crashes does not need a server restart — send it a message from a connected device and Claude Code serves it again (v2.1.238 or later).

Option 3 — a dedicated headless machine

A real upgrade, and the right answer if your only computer travels. It is just not the *first* move. Run it with no screen at all: SSH in over Tailscale or your LAN, start server mode inside `tmux`, and drive it from your phone.

Machine	Verdict
Mac mini (M-series, 16 GB+)	The pick if you are on macOS. Silent, idles at single-digit watts, runs the full dev toolchain, and happily runs headless. No dummy HDMI plug needed unless you want Screen Sharing at a sensible resolution.
Beelink / Minisforum / Intel NUC	The x86 Linux equivalent. Cheaper, and more RAM per dollar. Pick this if your toolchain is happier on x86 or you want to run containers without thinking about it.
An old laptop you already own	Free, already has a keyboard for recovery, and the battery is a built-in UPS. Genuinely the best value here. Close the lid and set the lid-switch handling from page 2.
Raspberry Pi 5	I would skip it. ARM plus limited RAM starts hurting the moment a real Node or Docker build shows up. Fine for a toy repo, frustrating for actual work.

Headless setup checklist

- Enable SSH — **macOS:** System Settings, General, Sharing, Remote Login. **Linux:** `sudo systemctl enable --now ssh`.
- Install **Tailscale** on the box and on your laptop and phone. Reaching it from anywhere with no port forwarding, no dynamic DNS, and nothing exposed to the internet is worth the ten minutes.
- Come back from a power cut by itself — **macOS:** `sudo pmset -a autorestart 1 sleep 0`. **Linux:** enable Restore on AC Power Loss in the BIOS.
- Sign in to Claude Code and accept the workspace trust dialog *interactively, over SSH*, in the project directory before you automate anything. This is the step people skip, and the service then fails on boot.
- Clone your repos and install the toolchain. Remote Control gives you the machine's real filesystem and its MCP servers — that is the whole point of it over a cloud session — so the box needs a working dev environment.

Windows: the headless build

Windows changes two things: which runtime you install Claude Code into, and which traps you hit keeping it up. The hardware question is the same one as Option 3, so this chapter also carries the prices.

First decision: WSL 2, not native Windows

Claude Code installs and runs natively on Windows from PowerShell or CMD, with Git for Windows as an optional extra that enables the Bash tool. For an unattended headless box, install it into WSL 2 anyway:

Runtime	Sandboxing	Notes
Native Windows	Not supported	Shell commands run through the PowerShell tool, or Git Bash if Git for Windows is installed.
WSL 2	Supported	Linux toolchain, native Linux paths, sandboxed command execution.
WSL 1	Not supported	Only if WSL 2 is unavailable on the machine.

`claude remote-control --sandbox` is exactly what you want on a machine you drive from a phone and cannot watch, and it works only under WSL 2. WSL also brings `tmux`, which Windows otherwise lacks entirely. This one choice is worth more than the whole price difference below.

Two builds

Tier	Machine	Price	Idle	Power/yr
Under \$1k	Beelink SER8 — Ryzen 7 8845HS (8C/16T), 32GB DDR5-5600, 1TB NVMe, 2.5GbE, Windows 11 Pro. Alternative: GMKtec K8 Plus at \$649, down from \$899.	\$499 sale \$619 list	10–14 W	~\$19
Over \$1k	Minisforum MS-A2 — Ryzen 9 9955HX (16C/32T, to 5.4 GHz), 64GB DDR5, 1TB NVMe, dual 10GbE SFP+, PCIe x16 slot. Barebone \$839.	\$1,103	20–25 W	~\$35

Power costed at the US average of 18.3 cents per kWh, running 24/7. The larger machine draws up to 100 W sustained, so it is also audible under load.

What the extra \$550 buys

	Under \$1k	Over \$1k	Matters here?
Cores	8C / 16T	16C / 32T	Only for heavy builds
Memory	32 GB	64 GB	32 GB is already plenty
Network	2.5GbE	Dual 10GbE SFP+	No — unless you run a NAS
Expansion	None	PCIe x16 slot	No — unless adding a GPU
Idle power	~12 W	~22 W	Costs you more
Noise	Quiet	Audible under load	Worse in a room you sit in

IS THE OVER-\$1K TIER WARRANTED? NO — NOT FOR THIS

The model runs on Anthropic's servers, not on your box. Unlike a local-LLM machine, a Claude Code host does no inference at all, and Claude Code's own stated requirement is 4 GB of RAM — 32 GB is already eight times that. An always-on session spends nearly its whole life idle, waiting for you to send a message. None of the larger machine's advantages, not the sixteen cores, the 10GbE, or the PCIe slot, do anything for that workload. Your box only does file I/O, git, and whatever your own builds and tests need.

Buy the bigger machine only if it is *also* becoming your homelab: a NAS on 10GbE, a Proxmox host, a GPU for local models, or genuinely heavy parallel compiles where sixteen cores halve your build time. Those are real reasons to want a server. They are not reasons to keep a session alive, and it is worth being honest with yourself about which one you are buying.

ON TIMING

A large part of that \$550 gap is simply the second 32 GB of DDR5 at 2026 memory prices. The SER8 takes two standard DDR5-5600 SO-DIMMs and officially supports 64 GB, so buy 32 GB now and add the rest when memory normalises. Paying the shortage premium for RAM you do not yet need is the worst version of this purchase.

Setting it up

Run the first block in an Administrator PowerShell on the box itself, or over SSH once that is enabled:

```
# Never sleep
powercfg /change standby-timeout-ac 0
powercfg /change monitor-timeout-ac 0
powercfg /hibernate off

# SSH in rather than RDP -- works on Home edition too
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
Set-Service -Name sshd -StartupType Automatic
Start-Service sshd

# WSL 2
wsl --install -d Ubuntu
```

Then inside Ubuntu, interactively the first time, so you can accept the workspace trust dialog and sign in:

```
sudo apt update && sudo apt install -y tmux
curl -fsSL https://claude.ai/install.sh | bash
cd ~/code/my-project && claude # /login, accept workspace trust
```

Start it at boot with Task Scheduler:

```
$cmd = "-d Ubuntu -u YOURUSER -e /bin/bash -lc `tmux new-session -d -s claude " +
        "'claude remote-control --continue --spawn worktree --sandbox'`""
Register-ScheduledTask -TaskName "ClaudeRemoteControl" `
    -Action (New-ScheduledTaskAction -Execute "wsl.exe" -Argument $cmd) `
    -Trigger (New-ScheduledTaskTrigger -AtLogOn -User "$env:COMPUTERNAME\$env:USERNAME")
```

To get back in later: `ssh user@box`, then `wsl`, then `tmux attach -t claude`.

THREE TRAPS THE MACOS AND LINUX SETUPS DO NOT HAVE

1. Windows Update reboots are the single biggest killer of a 24/7 Windows box. Set Active Hours and turn off "Restart as soon as possible". Pair it with auto-login (netplwiz, uncheck the password requirement) so the logon task actually fires after a forced restart — otherwise the machine boots to a login screen and the session never comes back. If auto-login bothers you, add a second logon task running `rundll32.exe user32.dll,LockWorkStation` to lock it immediately.

2. Use -AtLogOn, not -AtStartup. WSL needs a user context to start cleanly.

3. Skip RDP. It is Pro-only and unnecessary here. SSH plus `wsl` plus `tmux attach` is the whole remote story, and it works on any edition.

Option 4 — skip the hardware entirely

Cloud sessions at `claude.ai/code` run on cloud infrastructure with no local setup at all, so there is nothing to keep alive. Both surfaces use the same interface; the difference is purely where execution happens.

	Remote Control	Cloud session
Runs on	Your machine	Anthropic-managed infrastructure (or your org's self-hosted runners)

	Remote Control	Cloud session
Your files	Real local filesystem, and @ autocompletes real paths	A fresh clone in an ephemeral container
Local MCP servers, tools, project config	All available	Not available
Best for	Continuing local work from another device	Starting a task with no setup, a repo you have not cloned, or many tasks in parallel
Keeping it alive	Your problem — this whole guide	Not a concern

These are not exclusive, and the honest answer for most people is both: an always-on box for work that touches your own machine, cloud sessions for everything else.

Making it survive reboots

tmux survives a disconnect but not a restart, and server mode exits after a long network outage. A supervisor fixes both. On Linux:

```
# ~/.config/systemd/user/claude-rc.service
[Unit]
Description=Claude Code Remote Control
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
WorkingDirectory=%h/code/my-project
ExecStart=/usr/local/bin/claude remote-control --continue --spawn worktree
Restart=always
RestartSec=30

[Install]
WantedBy=default.target

systemctl --user daemon-reload
systemctl --user enable --now claude-rc
sudo loginctl enable-linger $USER      # <- runs without you being logged in
journalctl --user -u claude-rc -f     # watch it
```

The enable-linger line is the one people miss: without it the service stops the moment your login session ends. On macOS, the equivalent is a LaunchAgent:

```

<!-- ~/Library/LaunchAgents/com.user.claude-rc.plist -->
<?xml version="1.0" encoding="UTF-8"?>
<plist version="1.0"><dict>
  <key>Label</key>          <string>com.user.claude-rc</string>
  <key>ProgramArguments</key> <array>
    <string>/usr/local/bin/claude</string>
    <string>remote-control</string>
    <string>--continue</string>
  </array>
  <key>WorkingDirectory</key> <string>/Users/you/code/my-project</string>
  <key>RunAtLoad</key>        <true/>
  <key>KeepAlive</key>        <true/>
  <key>StandardErrorPath</key><string>/tmp/claude-rc.err</string>
</dict></plist>

```

```
launchctl load -w ~/Library/LaunchAgents/com.user.claude-rc.plist
```

If you would rather keep a terminal you can attach to and read, run `tmux` from the service instead of running `claude` directly: `tmux new-session -d -s claude 'claude remote-control --continue'`. You lose nothing and gain a scrollbar you can inspect over SSH.

When it breaks: a field guide

Symptom	Cause and fix
Session shows offline within seconds	The <code>claude</code> process exited — you closed the terminal, quit VS Code, or the SSH session ended without <code>tmux</code> . Bring it back with <code>claude --continue</code> or <code>claude --resume</code> , then run it under a supervisor.
Server mode process is simply gone	The machine was awake but offline for more than about ten minutes, so server mode gave up. Expected behaviour. This is what <code>Restart=always</code> is for.
"Could not reach the Remote Control server for about 30 minutes"	Presence heartbeats were failing while the rest of the connection held up. Run <code>/remote-control</code> to reconnect.
Disconnects after a VPN or network change	Something between you and Anthropic answered HTTP 403. Claude Code retries for up to three minutes; past that it disconnects and names what refused — a network edge, or a proxy, VPN, or firewall on your own network.
"Couldn't reconnect to your Remote Control session"	Possibly temporary — try again. If it persists, <code>/remote-control</code> starts a fresh session under the current login; your local session keeps running without Remote Control meanwhile.
Phone notifications are late or missing	iOS: Focus modes and notification summaries. Check Settings, Notifications, Claude. Android: exempt the Claude app from battery optimization.
Pushes fire while you are sitting at the machine	Expected while you are focused on the terminal. To extend that to any time you are at the machine, point <code>CLAUDE_CLIENT_PRESENCE_FILE</code> at a marker file and have a screen-lock listener create it on unlock and delete it on lock.
A command does nothing from my phone	Some commands are terminal-only, notably <code>/plugin</code> and <code>/resume</code> . Others need an argument instead of a picker: <code>/model sonnet</code> , <code>/effort high</code> .

One thing to know about data

While Remote Control is connected, the session transcript — your messages, Claude's responses, and tool activity — is stored on Anthropic servers. That is what keeps the conversation in sync across your devices and lets a session reconnect after a network drop. Execution and filesystem access stay on your machine, and stored transcripts are retained under the Data usage policy. Worth knowing before you point an always-on box at a sensitive repository.

What I would actually do

Tonight	Run Option 1 on whatever machine is already on your desk. Start it in tmux, disable sleep, and confirm end to end that you can pick the session up from your phone. Fifteen minutes including the QR code.
This week	Live with it. Notice what actually breaks. For most people the answer is "nothing" — the phone-to-desk handoff just works, and the question of hardware quietly goes away.
If one thing keeps breaking	And that thing is "my laptop went to a coffee shop", buy the mini PC. You will set it up in an hour, knowing exactly what you want from it, instead of guessing now.

The temptation with a question like this is to buy the interesting answer before confirming it is the necessary one. Server mode in tmux on a machine that does not sleep is, for the overwhelming majority of setups, indistinguishable from the dedicated box — and it costs nothing to find out which camp you are in.

Command cheat sheet

Command	What it is for
<code>claude --remote-control</code>	Normal interactive session that is also reachable from your phone and browser. Short form: <code>--rc</code> .
<code>/remote-control</code>	Turn on Remote Control for the session you are already in. Short form: <code>/rc</code> . Also the command to reconnect after any disconnect.
<code>claude remote-control</code>	Server mode: park and serve sessions on demand. The always-on shape.
<code>claude --continue</code>	Bring back the last conversation in this directory, reconnecting Remote Control if it was on.
<code>claude --resume</code>	Pick an earlier conversation from a list. Terminal only.
<code>tmux new -s claude</code>	Start a named tmux session. <code>Ctrl-b d</code> detaches, <code>tmux attach -t claude</code> returns.
<code>caffeinate -s -i CMD</code>	macOS: keep the machine awake for as long as CMD runs.
<code>sudo pmset -a sleep 0</code> <code>disablesleep 1</code>	macOS: never sleep, permanently.
<code>systemctl --user enable --now claude-rc</code>	Linux: start the supervised service and enable it at boot. Pair with <code>loginctl enable-linger \$USER</code> .

Command	What it is for
<code>wsl --install -d Ubuntu</code>	Windows: install WSL 2, the runtime that supports sandboxing.
<code>powercfg /change standby-timeout-ac 0</code>	Windows: stop the machine sleeping on AC power.
<code>claude update</code>	Update the CLI. The fix when a flag in this guide is rejected as unknown.

Sources — all from the Claude Code documentation, September 2026.

Remote Control: code.claude.com/docs/en/remote-control

Advanced setup (Windows, WSL): code.claude.com/docs/en/setup

Claude Code on mobile: code.claude.com/docs/en/mobile

Claude Code on the web: code.claude.com/docs/en/claude-code-on-the-web

Cross-session messaging: code.claude.com/docs/en/cross-session-messaging

Scheduled tasks: code.claude.com/docs/en/scheduled-tasks

Hardware prices were current in September 2026 and move fast, especially during the memory shortage. Hardware opinions, the service files, and the recommendation are mine, not the documentation's. Version-gated behaviour noted above reflects the docs as of the compile date; run `claude update` if a flag is rejected as unknown.